

Progress Report

Ayush Madhav Kumar

RL for Jac Code Generation — July 2026



JASECI

Goal & experimental setup

- **Goal:** a local model that writes correct Jac — compiles *and* prints the exact expected output.
- **Hardware:** Apple M5 Pro, 48 GB unified memory, MLX LoRA. Trains the 30B-A3B MoE (~3B active); every dense ≥ 14 B and larger MoE OOMs.
- **Models:** fresh Qwen3-Coder-30B-A3B vs. jac-qwen3coder (same base, already SFT+DPO'd on Jac).
- **Grader:** Jac compiler + runtime, **exact-stdout = pass**. No learned reward model anywhere.

Task families

- **Hole-fill** — 84 deterministic tasks mined from `this_is_jac`
- **Conversion** — `python`→`jac`
- **Graph walkers** — node/edge/walker (OSP) idiom
- **Synthetic** — 32 fresh deterministic tasks

Three eras of experiments

Era 0 — weekend GRPO

GRPO on both models,
STaR loop.

*“Supervised levers move
models; RL does not.”*

Jun 20–21

Era 2 — the correction

Eval *and* reward shared a
broken extractor.

Everything re-measured.

Jul 1–2

Jun 25–28

Era 1 — SFT ladder

Leak-free harness, 30-cell
ladder, tuned-GRPO arm.

“RL is a dead end.” (v1)

Every headline number from Eras 0–1 was measured wrong — a $\sim 3.5\text{--}4\times$ undercount.

Era 0 — weekend GRPO: everything at 14.3%

1. **Metal OOM** — group 6 / completion 512 blew 48 GB at iter 2; survived at group 4 / comp 256 (~ 38 GB peak).
2. $\sigma = 0$ **trap** — at 0% base pass every rollout scores the same $\Rightarrow$ advantage $(r - \bar{r})/\sigma = 0$, KL = 0.0 forever. **RL cannot bootstrap a skill the base has none of.**
3. **Splice bug** — models emit the whole `can ... {}` unit; the hole sat *inside* it $\Rightarrow$ nested broken Jac. Base, warm (SFT loss 0.006!), and GRPO scored identically — the splice was broken, not the models.
4. **Dense wall** — Qwen3.6-27B SFT OOM'd at iter 1 in every config; the whole line is inference-only on 48 GB.
5. **LoRA-GRPO null** — splice fixed, real reward variance (σ up to 0.11): +GRPO output **byte-identical** to its start point, KL ≈ 0 .

Phase 1 (31 tasks, holdout 7)	pass
jac-qwen3coder base	14.3%
+ GRPO	14.3%
qwen3coder base	0%
+ warm-start SFT	14.3%
+ GRPO	14.3%

Phase 2.5 (51 tasks, holdout 12): pass@1 = pass@8 = 14.3% for every arm. STaR flickered to pass@4 = 25% for one round, then died.

Era 1 — the SFT ladder, and a false verdict

Design (leak-free rebuild)

- Rungs: train- $N \in \{1, 3, 5, 10, 20, \text{all}\}$
- Conditions: base, SFT, SFT+GRPO, raw-GRPO control, tuned-GRPO (500 iters, $10\times$ LR)
- 2 models $\times$ 30 cells; corpus mined 51 $\rightarrow$ 84 deterministic tasks
- Sound splits, monotone reward, Wilson CIs

Result (on the broken eval)

- Greedy pass@1 **flat** in every cell:
 $\sim 26.7\%$ (66-corpus), 11.1% (84-corpus)
- The Jac specialist **frozen**: literally the same number in every row — 26.67 / 11.11 / 21.43%
- Tuned-GRPO = default GRPO, even with real variance (σ 0.09–0.21)
- SFT lifted pass@8 mean +5–7pp, never beat base's max

v1 verdict (Jun 28): “RL is a dead end” — declared done. All of it rested on the eval.

The measurement bug that faked the dead end

- `extract_jac/unwrap_unit` grabbed the **driver docstring** instead of the model's answer whenever the model echoed the whole driver $\Rightarrow$ auto-fail.
- The GRPO reward used the **same extractor** $\Rightarrow$ GRPO was trained *and* scored on garbage.
- Fixed in 8164ee2: brace-matched, name-targeted body extraction — then everything re-run.

Same models, same holdout ($n = 18$)

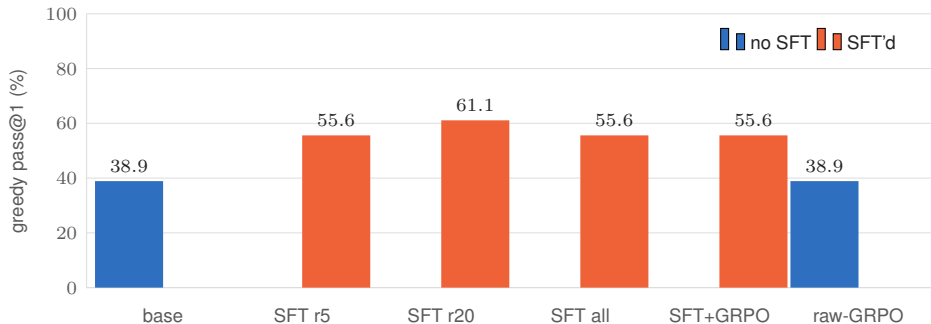
greedy pass@1	broken	fixed
fresh qwen3coder	11.1%	33.3%
jac-qwen3coder	11.1%	38.9%

$\approx 3.5 \times$ undercount

Lesson

Verify the grader before trusting a null result. One shared helper silently capped three weeks of experiments.

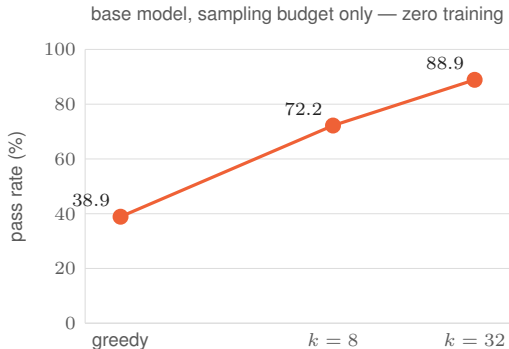
Corrected ladder — SFT works (pure-fn holdout, $n = 18$)



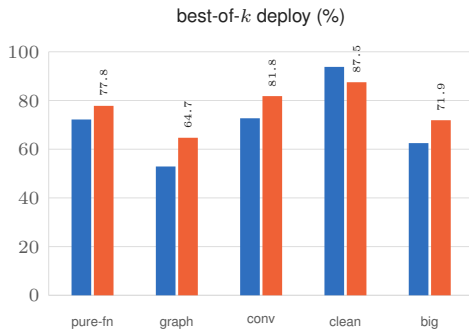
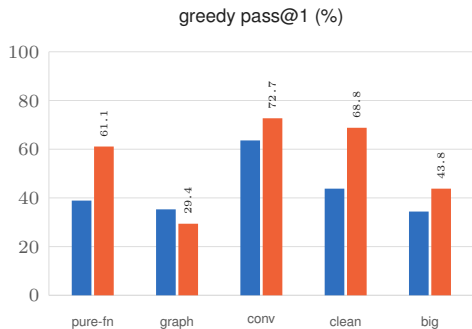
SFT lifts greedy **38.9** → **61.1%** (+22pp, sweet spot at rung-20; rung-all drops to 55.6 — one task regressed, task interference). GRPO $\approx$ SFT (55.6 either way, a *valid* null now). raw-GRPO from base = base — **GRPO can't bootstrap**. Best-of- k deploy: 72.2 → 77.8%.

The gap is syntax, and the compiler closes it for free

- jac-qwen3coder: pass@1 38.9% vs pass@8 66.7% — **+27.8pp gap**. The right Jac is *reachable*; greedy lands on a near-valid variant that won't compile.
- Failures are **compile-fails, not wrong answers** (missing ';', `here.jid` vs `jid(here)`).
- Fresh model: pass@1 = pass@8 = 33.3% — gap 0, a real dead end.
- The Jac compiler is a **free, perfect verifier**: sample k , keep the first that compiles. Deploy = oracle throughout.



Across all five holdouts



base SFT

Conversion is the peak: **72.7% greedy** / **81.8% best-of- k** — richer spec beats hole-fill by +9–11pp.

Graph walkers: greedy dips under SFT (noise) but best-of- k gains **52.9** → **64.7%**.

n : pure-fn 18 · graph 17 · conversion 11 · clean 16 · big+fresh 32

Follow-ups: ceiling, honest gap, generalization

E1 · true ceiling $\sim 94\%$

Drop 2 junk regex tasks ($n = 16$): base best-of- k **93.8%**. SFT sharpens greedy ($43.8 \rightarrow 68.8\%$) but *narrows* sampling — its best-of- k (87.5%) dips one task below base.

Exploit vs explore: single-shot $\rightarrow$ SFT; k -budget $\rightarrow$ base + best-of- k .

E2 · the honest gap

Free-form NL $\rightarrow$ Jac: **0/3 for both** the hole-fill-SFT and conversion-SFT models.

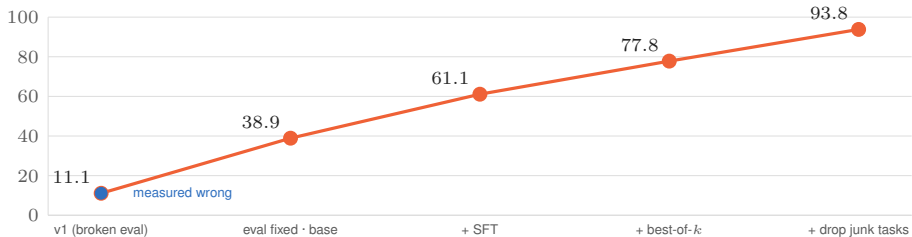
Not fixed by model choice — needs its own dataset. The one real remaining weakness.

E3 · SFT generalizes

Bigger, fresher holdout ($n = 32$, +16 synthetic): greedy $34.4 \rightarrow$ **43.8%**, best-of- k $62.5 \rightarrow$ **71.9%**.

Lift holds on tasks that didn't exist at training time (synthetic $4 \rightarrow 5$, library $7 \rightarrow 9$) — not memorization.

The journey: a bug hid an 11 → 94% generator



- **Verify the grader** before trusting a null — eval *and* reward shared the bug.
- The gap is **syntax, not capability** — compiled Jac is almost always exactly right.
- **Compiler = free perfect verifier** — best-of- k ships 78–94% with zero training.

- **SFT works & generalizes**; sweet spot ~ 20 tasks — more causes interference.
- **LoRA-GRPO can't bootstrap** and $\approx$ SFT after warm-start (matches Yue et al. 2504.13837).
- **Conversion beats hole-fill** +9–11pp — richer, unambiguous spec.

Shipped

- `rl/generate.py` — best-of- k generator: sample k at $T=0.8$, return the first candidate the Jac compiler accepts. Live-verified; deploy = oracle.
- Studio **GENERATE panel** + all corrected result charts.
- Studio **RL data pipeline** — SFT / GRPO / conversion / synthetic datasets and splits, browsable end-to-end.
- Broken-eval numbers flagged with banners across all docs.

Deployable today — no further training

pure functions (best-of- k)	~78%
conversion + SFT	82%
clean tasks	94%
at $k = 32$	89%

Next steps

- **Close the E2 gap:** SFT on free-form NL → Jac prompts — the one measured weakness.
- **Spec → Jac dataset for the graph-walker idiom** — conversion can't source it (no Python equivalent of OSP).
- **Curriculum / quality-filter the train pool** — kill the rung-all task interference (or keep training at ~ 20).
- **Distillation / expert iteration** — the one method that can expand the capability boundary, not just sharpen it.
- **Whole-file generation track** — bridge from hole-fill to generating full Jac programs; gets its own ladder.
- **Guardrails:** partial-credit grading; Wilson CIs on every claim — “gap closed” means SFT pass@1 CI reaches base pass@8.

Fixing the graph-walker weak spot

Graph holdout ($n = 17$)

	base	+SFT
greedy pass@1	35.3%	29.4%
best-of- k	52.9%	64.7%

Only family where SFT **hurts** greedy — the tell for **task interference**: the SFT mix is dominated by hole-fill/conversion, graph examples get outvoted.

Fix plan, priority order

1. **Dedicated graph-walker dataset** — spec→jac synthetic set for the OSP idiom (walker/node/edge/visit). Conversion structurally can't source this: Python has no walker equivalent.
2. **Curriculum-weighted SFT / per-family adapter** — fix the interference itself, not just add data; graph examples get proportional gradient signal instead of being outvoted by volume.
3. **Stopgap, zero training, ships today** — detect task type in `rl/generate.py`, bump $k \rightarrow 32$ specifically for graph-walker prompts.

#1 + #2 are the actual fix — #3 is a band-aid you can flip on right now.

Glossary — terms & context

- **Jac** — Jaseci's language; supersedes Python, adds graph-native OSP constructs.
- **OSP / graph walkers** — Object-Spatial Programming: node/edge/walker idiom; no Python equivalent.
- **SFT** — supervised fine-tuning on prompt→answer pairs.
- **DPO** — direct preference optimization; trains on chosen-vs-rejected pairs.
- **GRPO** — group-relative policy optimization (RL): sample rollout groups, advantage = $(r - \bar{r})/\sigma$.
- **LoRA** — low-rank adapters; only small matrices train. Whole pipeline: LoRA on MLX (Apple-silicon ML).
- **MoE / 30B-A3B** — mixture-of-experts, 30B params, ~3B active per token — why it trains on 48 GB.
- **warm-start** — brief SFT before RL so rollouts aren't all-fail.
- **STaR** — self-taught reasoner loop: sample, keep verified wins, SFT on them, repeat.
- **$\sigma = 0$ trap** — all rollouts score equal $\Rightarrow$ advantage 0 $\Rightarrow$ zero gradient; RL can't start from 0% skill.
- **KL** — divergence from start policy during RL; ≈ 0 = model didn't move.
- **greedy / pass@1** — one deterministic (temp=0) answer; the single-shot metric.
- **pass@ k (oracle)** — any of k samples correct; ceiling if a perfect verifier picked.
- **best-of- k (deploy)** — sample k , return first the compiler accepts; = oracle here.
- **holdout** — eval tasks never trained on; five sets ($n = 18/17/11/16/32$).
- **hole-fill** — task format: one function body blanked out of a working Jac file; exact-stdout grading.
- **conversion** — python→jac translation tasks; richest spec, best scores.
- **ladder / rung- N** — train on N tasks, eval on holdout; rungs 1→all isolate data-size effects.
- **task interference** — adding harder train tasks regresses learned ones (rung-20 → all: 61→56%).
- **Wilson CI** — binomial confidence interval; the noise bar on every pass rate.
- **expert iteration / distillation** — train on a stronger model's verified outputs; expands ability.